

Teórica 7.3

"Database System Concepts", A. Silberschatz, H. Korth, S. Sudarshan

Almacenamiento físico: Capítulo 12, de 12.1 a 12.3 y 12.6

Chapter 12 - Physical Storage Systems

12.1 - Overview of Physical Storage Media

We can classify the storage media by the *speed* with which data can be accessed, by the *cost* per unit of data, and the *reliability*.

Cache: the fastest and most expensive form of storage. It is relatively small.

Main memory: stores data that can be operated with. It is too small and expensive to store a whole database. It is volatile: the contents are lost whenever a power failure or system crash occurs.

Flash memory: unlike main memory, its data can remain stored even if power gets turned off → it is non-volatile.

Magnetic-disk storage: it is the primary medium for the long-term online storage of data. It is non-volatile. To access data stored on magnetic disk, the system must first move the data from disk to main memory, from where they can be accessed.

Optical storage: DVDs and Blu-rays.

Tape storage: used primarily for backup and archival data → which must be stored safely for a long time. It is the slowest medium because it is accessed sequentially from the beginning of the tape.

Higher levels are expensive, but fast. At the bottom level, the cost per bit decreases as well as access times.

Primary storage → cache + main memory.

Secondary storage → flash memory + magnetic disks.

Tertiary storage → magnetic tapes.

12.2 - Storage Interfaces

Magnetic disks as well as flash-based solid-state disks are connected to a computer system through a *high-speed interconnection*.

They can also be situated remotely and connected by a *high-speed network* to the computer.

12.3 - Magnetic Disks

12.3.2 - Performance Measures of Disks

Access time: time from when a read or write request is required to when data transfer begins. To access data on a sector of a disk, the disk's arm must move so that it is positioned over the correct track, and then it must wait for the sector to appear under it as the disk rotates. The time the arm takes to reposition is called **seek time**, which increases with the distance that the arm must move.

Average seek time: the average of the seek times, measured over a sequence of uniformly distributed random requests.

Rotational latency time: the time the head of the arm waits for the sector to be accessed to appear beneath it, once the head is over the desired track.

Average latency time: defined as half the time for a full rotation of the disk.

Data-transfer rate: rate at which data can be retrieved from or stored to the disk.

I/O operations per second (IOPS)

Mean time to failure (MTTF)

Requests for disk input or output are generated by the file system but can also be generated directly by the database system. Each request specifies the address on the disk to be referenced; that address is in the form of a *block number*.

A **disk block** is a logical unit of storage allocation. Its sizes typically range from 4KB to 16KB. The data is transferred between disk and main memory in units of blocks.

A sequence of requests for blocks from disk may be classified as a *sequential access pattern* or a *random access pattern*.

Sequential access: successive requests are for **successive block numbers**, which are on the same track or adjacent tracks in the disk. A *seek may be required* for the first block, but successive requests would either *not require a seek, or require a seek to an adjacent track*, which is faster than a seek to a track that is farther away. *Data transfer rates are highest with a sequential access pattern, since seek time is minimal.*

Random access: successive requests are for **blocks that are randomly located on disk**. *Each request would require a seek*. Since only a small amount (one block) of data are read per seek, the *data transfer rate is significantly lower* with a random access pattern.

12.6 - Disk-Block Access

Requests for disk I/O are generated by the database system. Each request specifies a *disk identifier* and a *logical block number* on the disk. Data are transferred between disk and main memory in units of blocks.

A number of techniques have been developed for *improving the speed of access to blocks*, by minimizing the number of accesses, and in particular minimizing the number of random accesses.

- **Buffering**: blocks that are read from disk are *stored temporarily in a buffer* to satisfy future requests.
- **Read-ahead**: when a disk block is accessed, *consecutive blocks from the same track are read* into a buffer even if there is no pending request for the blocks.
- **Scheduling**: If several blocks from a cylinder need to be transferred from disk to main memory, we may be able to *save access time by requesting the blocks in the order in which they will pass under the heads*. If the desired blocks are on different cylinders, it is *advantageous to request the blocks in an order that minimizes disk-arm movement*.
- **File organization**.
- **Non-volatile write buffers**.

Estructuras de almacenamiento: Capítulo 13, de 13.1 a 13.4

Chapter 13 - Data Storage Structures

13.1 - Data Storage Structures

Persistent data are stored on non-volatile storage → magnetic disks or SSDs. These are *block structured devices - data are read or written in units of a block*. Databases deal with records though, which are usually smaller than a block.

Databases use operating system files to store records since this means as an abstraction of the details of the underlying blocks. Since the database needs to *ensure efficient access* and *recover the records from failures*, they must be *aware of blocks*.

A database needs to:

- know *how to organize the records* in the file structure and *in which order*.
- know *how to organize data about the relational schemas* as well as *storage organization* in the data dictionary.

For a CPU to access data, it must be in main memory first, whereas persistent data must be resident on non-volatile storage such as magnetic disks or SSDs. For *databases that are larger than main memory*, data must be *fetched from non-volatile storage and saved back if it's updated*.

13.2 - File Organization

A database is mapped into different files that are maintained by the operating system. These files are kept permanently on disks. A **file** is *organized logically as a sequence of records*. These records are *mapped onto disk blocks*. Files are provided as a basic construct in operating systems, so we assume *there is an underlying file system*.

A file is also logically partitioned into blocks. A *block can contain several records* - assuming *no record is larger than a block*. Also, *each record is contained entirely in a single block*.

In a relational database, tuples of distinct relations are generally of different sizes.

Mapping database → files

- Use several files and to store records of only one fixed length in any given file.
- Structure the files so that multiple lengths of records can be stored.

Files of fixed-length records are easier to implement than are files of variable-length records.

13.2.1 - Fixed-Length Records

Consider a file of **instructor** records:

```
type instructor = record
    ID varchar (5);
    name varchar(20);
    dept_name varchar (20);
    salary numeric (8,2);
end
```

Assuming that each **char** occupies 1 byte and that **numeric (8,2)** occupies 8 bytes. Suppose that instead of allocating a variable amount of bytes for attributes **ID**, **name** and **dept_name**, I allocate the maximum number of bytes that each attribute can hold. Therefore, *the instructor record is 53 bytes long*.

I can use the first 53 bytes for the first record, the next 53 bytes for the second record, and so on...

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

There are two problems with this approach:

1. Unless the block size happens to be a multiple of 53 (unlikely), *some records will cross block boundaries*. One part of the record will be stored in one block and the other part will end up stored in another block. It would require two block accesses to read or write this record.

To avoid the problem, I allocate only as many records to a block as would fit entirely in the block. Any remaining bytes of each block are left unused.

2. It is *difficult to delete a record from this structure*. The space occupied by the record to be deleted *must be filled with some other record* of the file, or there must be a way of *marking deleted records so that they can be ignored*.

When a record is deleted, I could move the record next to it into the space that has been freed up, and repeat this for all the next records. This implies moving a huge number of records. It might be easier to move just the last record in the block to the newly freed space.

It is undesirable to move records to occupy the space freed by a deleted record, since doing so requires additional block accesses.

Since insertions are more frequent than deletions, it is just *fine to leave the free space after a record deletion just to wait for a potential insertion in the future* before occupying the space right away.

A simple marker on a deleted record is not enough, since it is hard to find this available space when an insertion is being done.

Therefore, a number of bytes will be allocated at the beginning of the files as a **file header**. It will contain a *variety of information about the file*.

For now, there is the necessity to *store the pointer* (address) of the *first deleted record*.

The *first record will contain the address of the second free record*, and so on.

This linked list of free records form a **free list**.

To insert a new record, I have to *use the record pointed to by the header*. I also have to change the header pointer to point to the next available record. If *no space is available, we add the new record to the end of the file*.

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000



Insertion and deletion for files of fixed-length records are *simple to implement* because *the space made available by a deleted record is exactly the space needed to insert a record*.

13.2.2 - Variable-Length Records

They are needed since many databases include *strings*, a variable length field.

Different techniques exist to handle these records, and there are two common problems that must be solved by them:

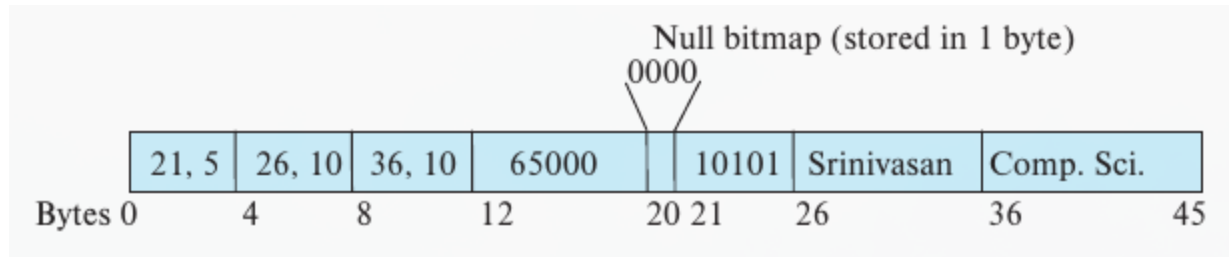
1. *How to represent a single record* in such a way that individual attributes can be extracted easily, even if they are of variable length.
2. *How to store variable-length records within a block*, such that records in a block can be extracted easily.

A representation of a record with variable-length attributes includes an *initial part of fixed-length information* followed by the *contents of variable-length attributes*.

Fixed-length attributes (numeric values, dates) are *allocated as many bytes as required to store their value*.

Variable-length attributes (**VARCHAR**) are *represented in the initial part of the record by a pair (OFFSET, LENGTH)*; where **OFFSET** denotes where the data for that attribute begins within the record, and **LENGTH** is the length in bytes of the variable-sized attribute.

The values for the variable-length attributes are stored *consecutively, after the initial fixed-length part of the record*. Thus, the initial part of the record stores a fixed size of information about each attribute, whether it is fixed-length or variable-length.

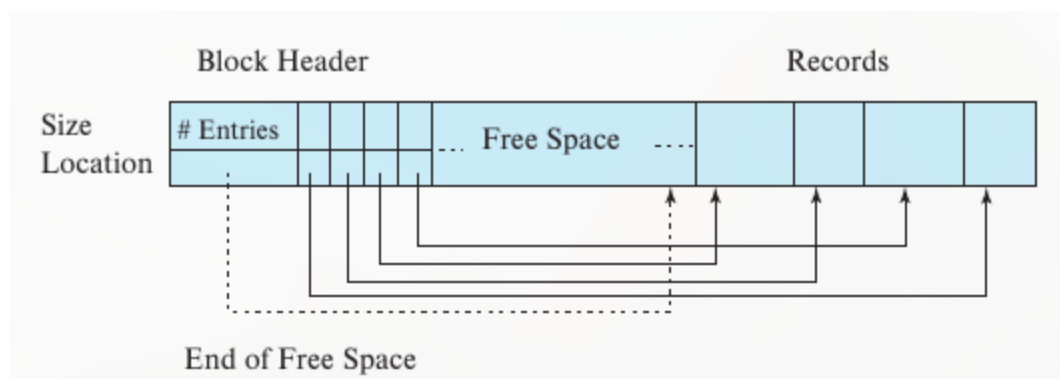


This shows a record of **Instructor** in which its first three attributes **ID**, **name** and **dept_name** are variable-length strings; and its fourth attribute **salary** is a fixed-size number.

The figure depicts that the offset and length values are stored in two bytes each, for a total of 4 bytes per attribute. The **salary** attribute is stored in 8 bytes, and each string takes as many bytes as it has characters.

The figure also illustrates the use of a **null bitmap**, which indicates *which attributes of the record have a null value*. In this particular record, if the **salary** were null, the fourth bit of the bitmap would be set to 1, and the salary value stored in bytes 12 through 19 would be ignored.

The **slotted-page structure** is commonly used for organizing records within a block and it looks like this:



There is a header at the beginning of each block, containing (1) the *number of record entries in the header*, (2) the *end of free space in the block*, and (3) an *array* whose entries contain the *location and size of each record*.

The actual records are *allocated contiguously in the block*, starting from the end of the block. The *free space in the block is contiguous between the final entry in the header array and the first record*.

If a record is *inserted*, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header.

If a record is *deleted*, the space that it occupies is freed, and its entry is set to deleted.

13.2.3 - Storing Large Objects

Nothing relevant in here

13.3 - Organization of Records in Files

A relation is a set of records. Given a set of records, there are multiple ways to organize them in a file.

- **Heap file**: any record *can be placed anywhere* in the file *where there is space* for the record. There is *no ordering of records*. Typically, there is either a single file or a set of files for each relation.
- **Sequential file**: records are *stored in sequential order*, according to the value of a "search key" of each record.
- **Multitable clustering file**: records of several different relations are stored in the same file, and in fact in the same block within a file, to reduce the cost of certain join operations.
- **B+-tree file**: can provide *efficient ordered access to records* even if there are a large number of insert, delete, or update operations. It also supports very efficient access to specific records, based on the search key.
- **Hashing file**: a *hash function is computed on some attribute of each record*. The result of the hash function specifies in which block of the file the record should be placed.

13.3.1 - Heap File Organization

In a **heap file organization**, *a record may be stored anywhere in the file* corresponding to a relation.

When a record is inserted in a file, one option for choosing the location is to *always add it at the end of the file*. However, *if records get deleted*, it makes sense to *use the space thus freed up to store new records*. A database system must find these blocks as efficiently as possible.

Databases often use a space-efficient data structure called a **free-space map** to track which blocks have free space to store records.

To find a block to store a new record of a given size, the *database can scan the free-space map to find a block that has enough free space to store that record*. If there is no such block, a new block is allocated for the relation.

13.3.2 - Sequential File Organization

A **sequential file** is designed for *efficient processing of records in sorted order based on some search key*.

A **search key** is any attribute or set of attributes; it need not be the primary key, or even a superkey. To *permit fast retrieval of records in search-key order, we chain together records by pointers*. The pointer in each record *points to the next record in search-key order*. Furthermore, to minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

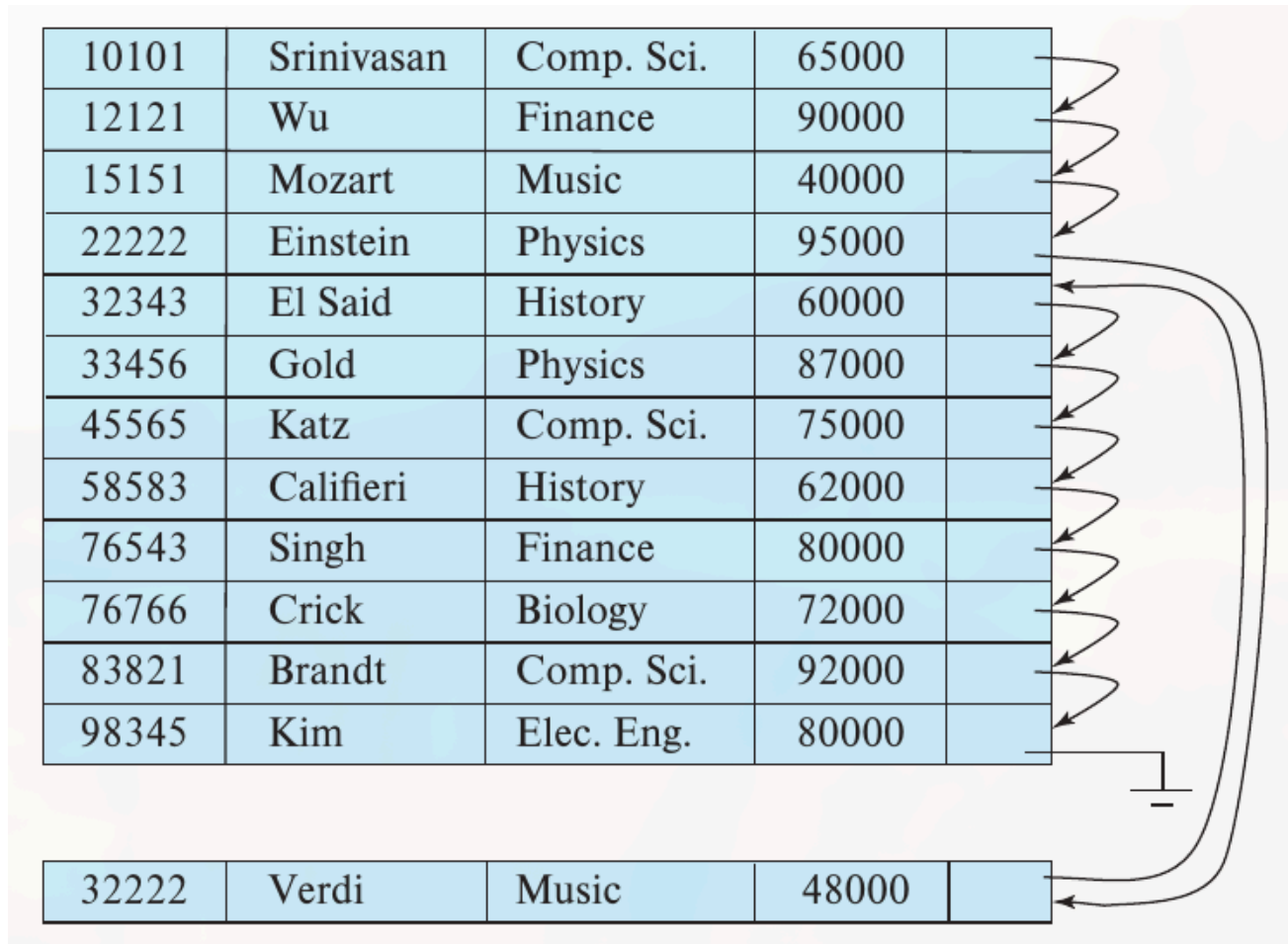


The sequential file organization *allows records to be read in sorted order*. The order is based on the search-key, which is **ID**.

It is difficult to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion. To manage deletions, the database uses **pointer chains** that follows two rules:

1. Locate the record in the file that comes before the record to be inserted in search-key order.

2. If there is a free record (space left after a deletion) within the same block as this record, insert the new record there. Otherwise, insert the new records in an overflow block. In either case, adjust the pointers so as to chain together the records in search-key order.



This is what happens if (32222, Verdi, Music, 48000) is inserted. This insertion is *done quickly*, but *forces sequential file-processing* applications to process records in an order that does not match the physical order of the records.

If relatively few records need to be stored in overflow blocks, this approach works well. However, *the correspondence between search-key order and physical order may be totally lost over a period of time, in which case sequential processing will become much less efficient*. At this point, the file should be reorganized, but reorganizing is very expensive.

13.3.3 - Multitable Clustering File Organization

Databases store each relation in a separate file or files. Each file and each block end up storing records of only one relation.

In some cases *it can be useful to store records of more than one relation in a single block*.

For example:

```
SELECT dept_name, building, budget, ID, name, salary
FROM department NATURAL JOIN instructor;
```

For each tuple of **department**, the system must locate the **instructor** tuples with the same value for **dept_name**. Regardless of how these records are located, however, they need to be transferred from disk into main memory. In the worst case, each record will reside on a different block, forcing to do one block read for each record required by the query.

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

Figure 13.9 The *department* relation.

No se encontró "instructor9.png".

All the instructor tuples for a particular **dept_name** are stored near the **department** tuple for that **dept_name**. These two relations *are clustered on the key dept_name*.

This structure works for *efficient processing of the join*. When a tuple of the **department** relation is read, the *entire block containing that tuple is copied from disk into main memory*. Since the corresponding **instructor** tuples are stored on the disk *near* the **department** tuple, the block containing the **department** tuple *contains tuples* of **instructor** needed to process the query. If **department** has so many instructors that the **instructor** records do not fit in one block, the remaining records appear on nearby blocks.

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Figure 13.11 Multitable clustering file structure.

This is a **multitable clustering file** that *stores related records of two or more relations in each block*.

The **cluster key** is the *attribute that defines which records are stored together* (in this case it is `dept_name`).

It can speed up certain join queries, but can slow down the processing of other queries.

13.3.4 - Partitioning

Consists of *partitioning the records in a relation into smaller relations stored separately*. This is done *on the value of an attribute* typically.

For example, the records in a `transaction` relation may be partitioned by year into smaller relations corresponding to each year, therefore creating the relations `transaction_2018`, `transaction_2019`...

This happens with a query like this:

```
SELECT *  
FROM transaction  
WHERE year=2018
```

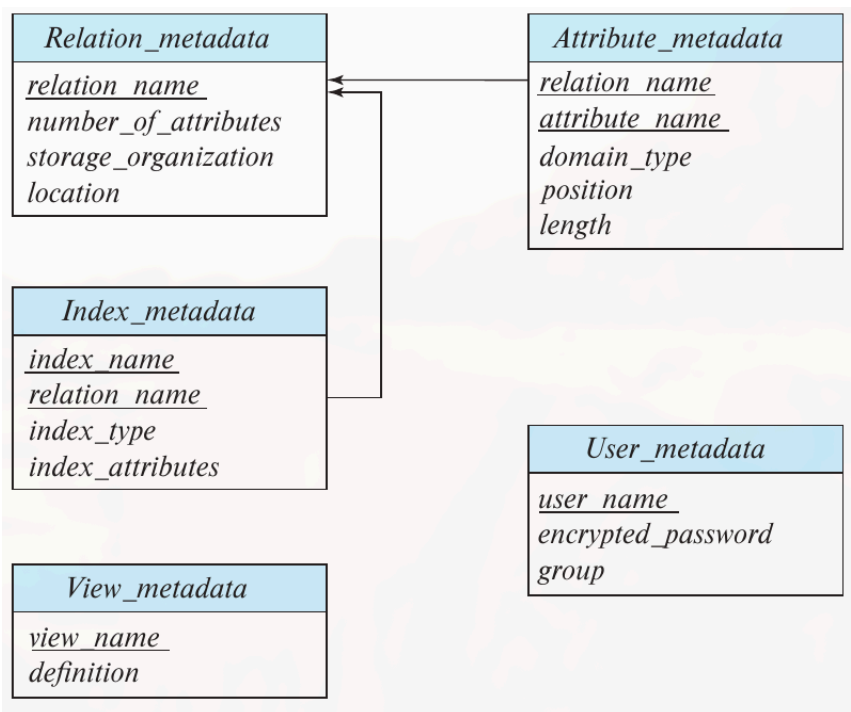
in which the database system would only access the relation `transaction_2018`, ignoring all the other relations, while a query without the selection condition would read all the relations.

13.4 - Data-Dictionary Storage

A database system must not only maintain data about the relations, but also about *the whole schema of the relations*. This "data about data" is the **metadata**, which is stored in the **data dictionary**.

It stores the name of the relations, name of attributes of each relation, domains and lengths of the attributes, constraints...

All this metadata creates a miniature database.



Índices: Capítulo 14, de 14.1 a 14.5

Chapter 14 - Indexing

14.1 - Basic Concepts

An **index** for a file in a database system works in much the same way as the index in a book.

Database system indices play the same role as book indices in libraries.

For example, to retrieve a **student** record given an **ID**, the database system would look up an index to find on which block the corresponding record resides, and then fetch the block, to get the appropriate **student** record.

Indices are *critical for efficient processing of queries in databases*. Without indices, every query *would end up reading the entire contents of every relation* that it uses.

Implementing an index on the **student** relation by keeping a sorted list of students' **ID** would not work well on large databases since the index would be huge, it would still take relevant time to search the student, and keeping this index sorted after insertions and remotions would be expensive.

For these reasons, there are two kinds of indices:

1. **Ordered indices**. Based on a sorted ordering of the values.

2. **Hash indices**. Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a **hash function**.

No technique is the best, so each technique must be evaluated based on the following criteria:

- **Access types**: *types of access that are supported efficiently*. Access types can include finding records with a *specified attribute value* or whose *values fall in a specified range*.
- **Access time**: time it takes to *find a particular data item* using the technique in question.
- **Insertion time**: time it takes *to insert a new data item* and to *update the index structure*.
- **Deletion time**: time it takes *to delete a data item*.
- **Space overhead**: the *additional space occupied by an index structure*.

Having more than one index might be desirable. For example, to search a book by author, subject or title.

Recall *an attribute or set of attributes* used to *look up records in a file* is called a **search key**. This definition differs from that one used in a primary key, candidate key and superkey. Using the notion of a search key, *if there are several indices on a file, there are several search keys*.

14.2 - Ordered Indices

An **index structure** is needed to *gain fast random access to records* in a file. Each index structure is *associated with a particular search key*.

An **ordered index** *stores the values of the search keys in sorted order* and *associates with each search key the records* that contain it.

The records in the indexed file may be stored in a sorted order (like books in a library). A file may have more than one index, on different search keys.

If the file with the records is *sequentially ordered*, a **clustering index** is an index whose *search key also defines the sequential order of the file*. The search key of these indices is usually the primary key (**PK**), but not always.

If the *search key of an index specifies an order different from the sequential order of the file*, then that is a **nonclustering index**.

Let's assume all files are ordered sequentially on some search key. These files with a *clustering index on the search key*, are called **index-sequential files**.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



This is a sequential file of **instructor** records. The records are stored in sorted order of **ID** (first column), which is the search key.

14.2.1 - Dense and Sparse Indices

Index entry → consists of a *search-key value and pointers to one or more records with that value as their search-key value*. The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.

There are two types of ordered indices:

- **Dense index**: *an index entry appears for every search-key value in the file.*
 In a *dense clustering index*, the index record contains the search-key value and a pointer to the first data record with that search-key value.
 In a *dense nonclustering index*, the index must store a list of pointers to all records with the same search-key value.

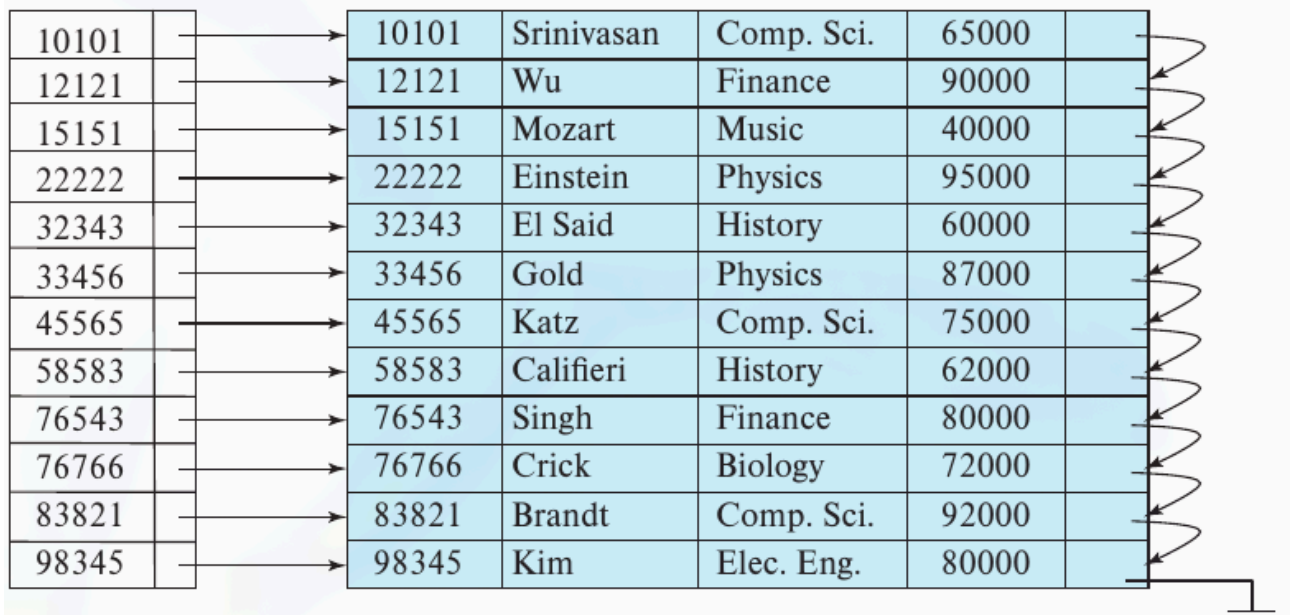


Figure 14.2 Dense index.

Suppose we are looking up the record of instructor with **ID = 22222**. Using this index, we follow the pointer directly to the desired record. Since **ID** is a primary key, there exists only one such record and the search is complete.

- **Sparse index**: *an index entry appears for only some of the search-key values*. Sparse indices can be used only if the relation is stored in sorted order of the search key (if the index is a clustering index).

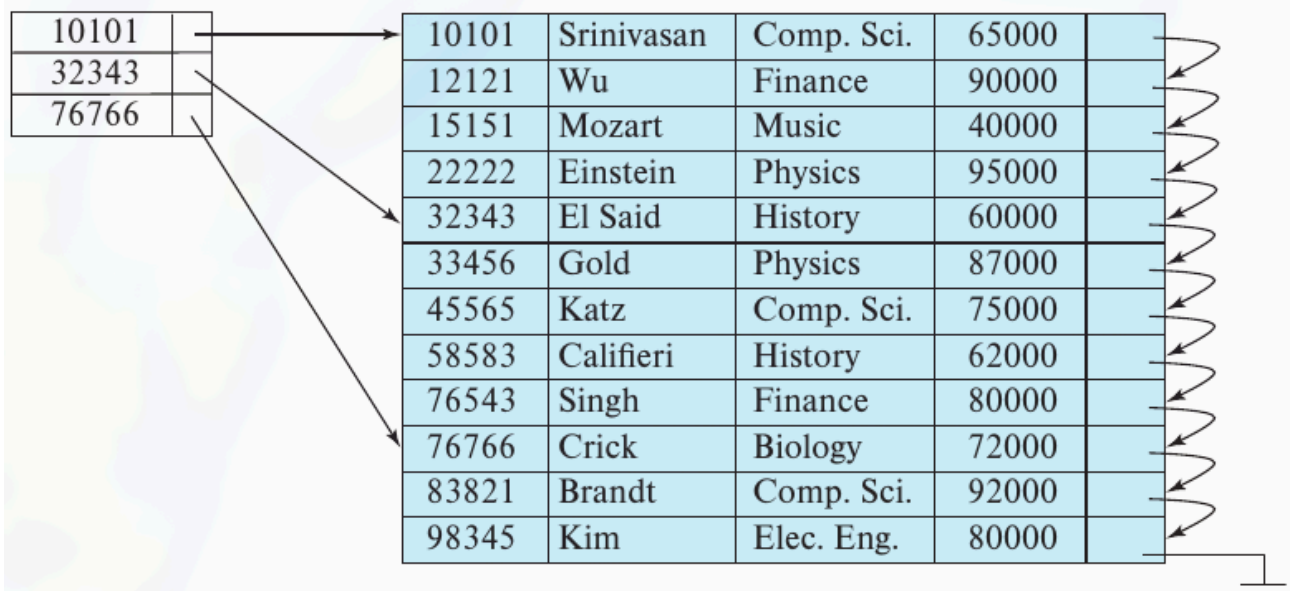


Figure 14.3 Sparse index.

Following the example, we do not find an index entry for **ID = 22222**. Since the last entry (in numerical order) before **ID = 22222** is **ID = 10101**, we follow that pointer. We then read the instructor file in sequential order until we find the desired record.

Consider a dictionary, in which the header of each page lists the first word alphabetically on that page. The words at the top of each page of the book index together form a sparse index on the contents of the dictionary pages.

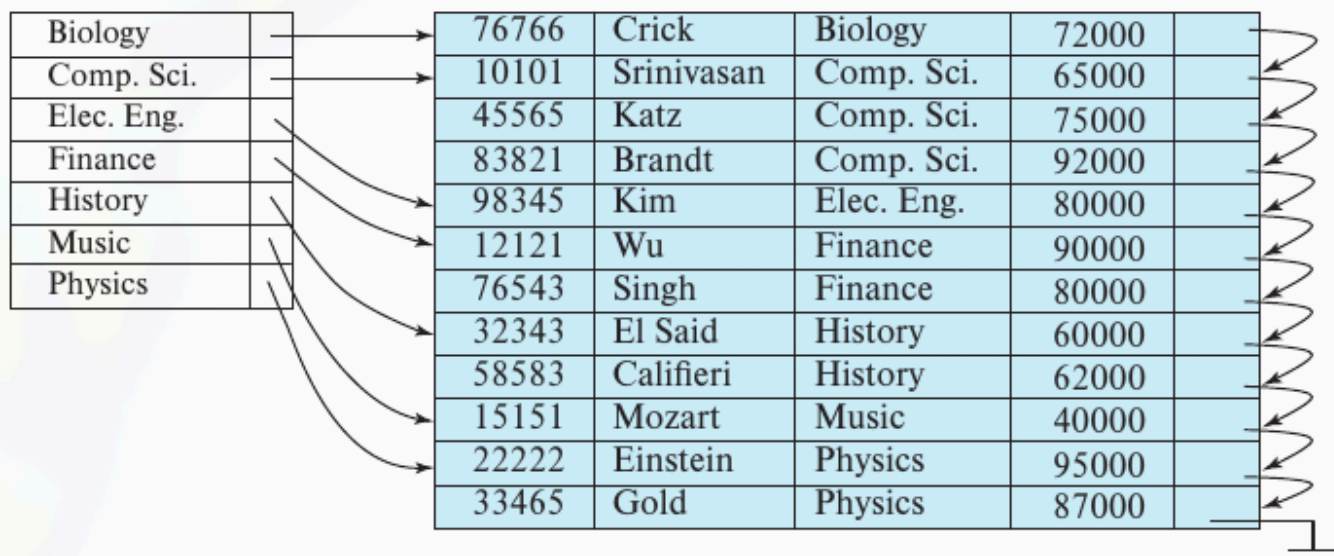


Figure 14.4 Dense index with search key *dept_name*.

In here, the search-key value is not a primary key. This image shows a dense clustering index for the **instructor** file with the search key being **dept_name**. The **instructor** file is sorted on the search key (**dept_name**) instead of **ID**. Otherwise, the index on **dept_name** would be a nonclustering index.

- We follow the pointer directly to the first **Department = History** record.
- We process this record and follow the pointer in that record to locate the next record the search-key (**dept_name**) order.
- We continue processing records until we find a record for a department other than **History**.

It is generally *faster to locate a record if we have a dense index rather than a sparse index*. But *sparse indices have advantages* over dense indices in that (1) they require *less space* and (2) they *need less maintenance* overhead for insertions and deletions. There is a trade-off between access time and space overhead.

14.2.2 - Multilevel Indices

We build a dense index on a relation with 1.000.000 tuples.

Index entries are smaller than data records, so we assume that 100 index entries fit on a 4KB block.

Our index occupies 10.000 blocks, or 40MB of space.

If the relation instead had 100.000.000 tuples, the index would instead occupy 1.000.000 blocks, or 4GB of space.

Such large indices are stored as sequential files on disk.

If an index is *small enough* to be kept entirely in main memory, the *search time to find an entry is low*. However, if the index is so *large* that not all of it can be kept in memory, *index blocks must be fetched from disk* when required.

So, the search for an entry in the index then requires several disk-block reads.

Binary search can be used on the index file to locate an entry, but the search still has a large cost.

To deal with this problem, the database system will *treat the index just as any other sequential file*, and it *creates a sparse outer index on the original index*, now called **inner index**.

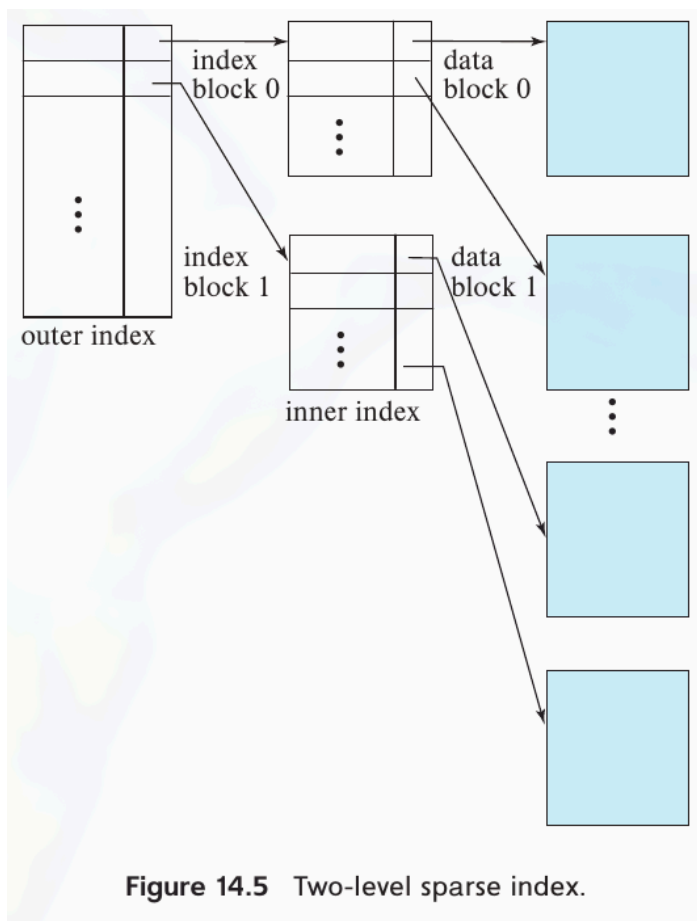


Figure 14.5 Two-level sparse index.

The index entries are always in sorted order, allowing the outer index to be sparse.

To locate a record:

1. The system has to *perform a binary search on the outer index to find the record for the largest search-key value less than or equal to the one that is desired.*
2. The *pointer points to a block of the inner index.*
3. This *block has to be scanned* until the record with the largest search-key value less than or equal to the one that is desired is found.
4. The *pointer in this record points to the block* of the file that *contains the desired record.*

An inner index with 10,000 blocks would require 10,000 entries in the outer index, which would occupy just 100 blocks. If we assume that the outer index is already in main memory, we would read only one index block for a search using a multilevel index, rather than the 14 blocks we read with binary search.

If our file is extremely large, even the outer index may grow too large to fit in main memory. With a 100.000.000-tuple relation, the inner index would occupy 1.000.000 blocks, and the outer index would occupy 10.000 blocks, or 40MB. Since there are many demands on main memory, it may not be possible to reserve that much main memory just for this particular outer index.

In that case, there is a need to create another level of index. This process can be repeated as many times as necessary. *Indices with two or more levels* are called **multilevel indices**. Searching for records with a multilevel index *requires significantly fewer I/O operations* than searching by binary search.

14.2.3 - Index Update

Every index must be updated whenever a record is *either inserted into or deleted* from the file. Further, in case a record in the file is updated, any index whose search-key attribute is affected by the update must also be updated.

14.2.3.1 - Insertion

The system performs a lookup using the search-key value that appears in the record to be inserted.

The actions the system takes depend whether the index is dense or sparse:

- **Dense indices:**
 1. If the search-key value does not appear in the index, the system inserts an index entry with the search-key value in the index at the appropriate position.
 2. Otherwise:
 1. If the index entry stores pointers to all records with the same search-key value, the system adds a pointer to the new record in the index entry.
 2. Otherwise, the index entry stores a pointer to only the first record with the search-key value. The system then places the record being inserted after the other records with the same search-key values.
- **Sparse indices:** (assuming the index stores an entry for each block). If the system *creates a new block*, it *inserts the first search-key value* (in search-key order) appearing in the new block into the index.
On the other hand, if the *new record has the least search-key value in its block*, the system *updates the index entry pointing to the block*; if not, the system *makes no change* to the index.